

Design and Analysis of Algorithms 2025/26 – Project 2

Felipe Magno (Num. 67994) and Theo da Silva (Num. 68548)

NOVA School of Science and Technology – Universidade NOVA de Lisboa

1 Problem Resolution

To start, in terms of data structures for our solution, we decided to use a two-dimensional array to store the positions of the beams. Moreover, we decided to use a graph implementation with adjacent lists (successors and predecessors). In terms of algorithms, we decided to implement a BFS search to filter any unnecessary nodes to the solution. Furthermore, Kahn's algorithm (topological sort) with built in acyclicity checking is used, so that disasters and beam removal order could be determined.

Firstly, we decided to incorporate a two dimensional array before beginning any operations with graphs or nodes. This was done so that beam dependancies could more easily and intuitively be made. The map is filled by iteration through every beam and filling the map based on the direction and length of the beam, filling up all coordinates that the beam belongs to with its integer ID.

0	0	6	0	0	0	0	0
0	5	6	0	4	0	0	0
0	5	1	1	4	0	0	0
0	5	0	3	2	2	2	2
0	0	0	3	0	0	0	0

Fig. 1. Filled in two dimensional array using problem description example

After the map has been filled, we iterate through each beam, starting from where it ends and checking the tiles in that direction until the end of the map. For every unique number it passes, which blocks the exit path, a new edge is added to the graph corresponding to the dependancy it has on other beams. For the graph implementation itself, we decided to use an adjacent list both with successor and predecessor data structures as both inAdjacentNodes and outAdjacentNodes need to be performed on BFS and Topological Sorting. This decision makes both these operations more efficient temporally.

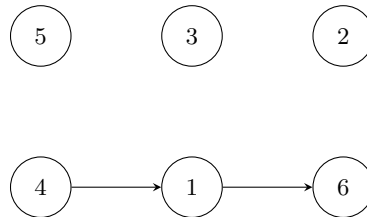


Fig. 2. Dependency graph of the problem description example

Subsequently, now that we have the graph to work with, we can process the graph using Breadth First Search and Topological Sort to get our final answer. We use BFS to find the nodes that necessarily need to be processed (beams to remove) given the columns you want to clear. The backward traversal through predecessors is done by executing `bfsExplore()` on any unfound nodes that belong to the columns you want to clear.

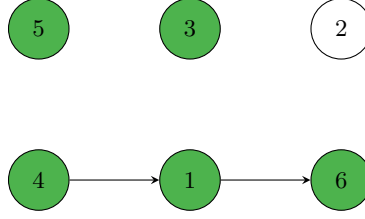


Fig. 3. Nodes processed by BFS (explored nodes are in green).

Once BFS is executed, we have a boolean array holding information about the nodes found, with the index of the array associating to if the node of that ID was processed by BFS. An int array is also generated that holds the ids of the nodes that were found, this is used to iterate through the Topological Sort algorithm more efficiently as to process only relevant nodes, instead of all of them.

After BFS is executed, the Topological Sort algorithm is executed to remove the beams in the right order. Since the problem states that whenever multiple beams can be freed, the one with the lowest identifier must be freed, a priority queue was used.

The final solution for the problem description example would then be:

3 4 1 5 6

2 Notation and Variable Definitions

To analyze the complexity of the algorithm, the following letters represent specific quantities involved in the process:

- R : The number of rows in the grid.
- C : The number of columns in the grid.
- B : The total number of beams provided in the input.
- K : The number of specific columns chosen to be cleared.
- D : The maximum dimension of the grid, defined as $\max(R, C)$.
- E : The number of graph edges created representing dependencies between beams.
- F : The number of beams that must be removed or reached by BFS ($F \leq B$).

3 Time Complexity

- **Setup and Mapping:** Allocating the grid map and marking cells occupied by beams takes $O(RC + B \cdot D)$.
- **Graph Construction:** Scanning from each beam's exit to find blockers takes $O(B \cdot D)$ plus $O(E)$ for edge insertions.
- **Target Search:** Scanning the K chosen columns across R rows and running a BFS through the graph takes $O(KR + F + E)$.
- **Topological Sort:** Removing F beams using a PriorityQueue for ordered results takes $O(F \log F + E)$.

Resulting Time Complexity:

$$O(RC + B \cdot D + KR + F \log F + E)$$

4 Space Complexity

- **Grid Storage:** The `beamIdMap` requires $O(RC)$ space to store the state of every cell.
- **Beam and Graph Records:** Storing B beam objects and their E dependency edges requires $O(B + E)$.
- **Auxiliary Arrays:** Arrays for tracking in-degrees and BFS visitation are bounded by $O(B)$.

Resulting Space Complexity:

$$O(RC + B + E)$$

In the worst-case scenario with a dense graph ($E = O(B^2)$), this simplifies to $O(RC + B^2)$.

5 Conclusion

5.1 Strengths

The solution has the following strengths:

- We used records such as `Beam`, `Answer`, and `FindRemove` to keep data representation simpler and to be able to pass several values through the algorithms.
- The grid `beamIdMap` allows constant-time lookup of which beam occupies a given cell.
- Adding 1 to `beamId` before setting their position in `beamIdMap`, allows us to use the value 0 to identify the empty spaces. That way we don't have to set the whole array to (eg. -1) before inserting the beamIds. Before returning the result we subtract 1 from the `beamId` to return the correct value.
- The array `found` allows constant-time lookup of which beams have to be removed, by setting `found[beamId] = true` on the beams that must be removed.
- The array `toRemove` allows ease of access of JUST the beams that have to be removed from the grid. It is used alongside with `amountFound`, so that we don't look further than we need in this array.
- BFS is used to find only the beams that can affect the chosen columns.
- Kahn's algorithm is used to compute a valid removal order.
- Cycles are detected naturally: if not all required beams can be removed by the topological sort, the result is `Disaster`.
- We use a `PriorityQueue` correctly match the specification of removing beams in ascending order.

5.2 Weaknesses

The main weakness of the solution is its memory usage. The algorithm allocates a full grid:

$$\text{beamIdMap} \in O(R \cdot C)$$

where R is the number of rows and C is the number of columns. This is efficient for direct access, but it can be expensive when the grid is large and sparse.

Another weakness is the possible size of the graph. In the worst case, many beams can block many other beams, producing up to:

$$E = O(B^2)$$

edges, where B is the number of beams.

Nota: We used Claude to help us format the visual representations with the tikz package.